

INFORMATION TECHNOLOGY INSTITUTE

9-Month Professional Diploma Program

E-Commerce Data Warehouse

End-to-End Big Data Pipeline

Bronze → Silver → Gold Architecture

Graduation Project Report

Big Data Track | April 2026

Executive Summary

This report presents a comprehensive end-to-end Big Data pipeline developed as the graduation project for the Information Technology Institute (ITI) 9-Month Professional Diploma. The project demonstrates the design, implementation, and validation of a modern Data Lakehouse architecture applied to a real-world Brazilian e-commerce dataset (Olist).

The pipeline ingests raw transactional data from a MySQL relational database, transfers it to HDFS, and processes it through three structured quality layers — Bronze, Silver, and Gold — using Apache Spark (PySpark) for transformation and Apache Hive for schema governance and analytical querying.

The Gold layer implements a star-schema dimensional model comprising one fact table (`fact_sales`) and four dimension tables (`dim_customers`, `dim_products`, `dim_reviews`, `dim_time`), making the data analytics-ready for Business Intelligence tools such as Power BI and Tableau.

Tables Processed	Pipeline Layers	Gold Tables	Storage Format
5 Source Tables	3 (Bronze/Silver/Gold)	1 Fact + 4 Dimensions	Apache Parquet

Table of Contents

- 1. Introduction & Project Background 4
- 2. System Architecture Overview 5
- 3. Technology Stack 6
- 4. Source Data & Database Design 7
- 5. Data Ingestion Layer (Bronze) 8
- 6. Data Cleaning Layer (Silver) 9
- 7. Analytical Layer (Gold) 11
- 8. Hive Schema & External Tables 13
- 9. Data Quality Findings 14
- 10. Key Business KPIs 15
- 11. Conclusion & Future Work 16

1. Introduction & Project Background

1.1 Project Overview

The exponential growth of e-commerce transactions generates massive volumes of structured and semi-structured data daily. Transforming this raw operational data into actionable business intelligence requires a robust, scalable data engineering pipeline. This project addresses that challenge by building a production-grade Data Lakehouse on Hadoop infrastructure, capable of ingesting, cleaning, transforming, and serving analytical data at scale.

1.2 Objectives

- Design and implement a 3-layer Medallion architecture (Bronze → Silver → Gold)
- Ingest MySQL relational data into HDFS using both Sqoop and manual transfer methods
- Apply data quality rules using PySpark including null handling, deduplication, and type casting
- Build a dimensional star schema optimised for BI reporting and analytics
- Expose the Gold layer via Apache Hive external tables for SQL-based querying
- Document all pipeline steps with reproducible SQL and PySpark scripts

1.3 Dataset

The Brazilian E-Commerce Public Dataset by Olist (available on Kaggle) serves as the data source for this project. It contains real-world transactional records from a multi-seller marketplace, covering the period 2016–2018 with over 100,000 orders.

Table	Primary Key	Description
customers	customer_id	Customer profile and location data
orders	order_id	Order lifecycle with timestamps
order_items	order_id + item_id	Line items: product, seller, price
reviews	review_id	Customer satisfaction scores & comments
products	product_id	Product catalogue with dimensions

2. System Architecture Overview

The pipeline follows the Medallion Architecture pattern, a widely adopted design in modern Data Lakehouse systems. Each layer represents a progressively higher level of data quality and analytical readiness.



2.1 Infrastructure

- Apache Hadoop (HDFS + YARN) running inside Docker containers
- Apache Hive Metastore for schema registry and table definitions
- Apache Spark (PySpark) for distributed data transformation
- MySQL 8.0 for relational source data, containerised via Docker
- Apache Sqoop for JDBC-based data transfer (alternative path also documented)

2.2 Data Flow

Raw CSV files are first loaded into MySQL then exported to HDFS. PySpark reads the raw data, applies quality rules, and writes clean Parquet files. Hive external tables are created pointing to the HDFS paths, enabling SQL access without moving data.

3. Technology Stack

Technology	Version / Tool	Role in Pipeline
Apache Hadoop	HDFS + YARN	Distributed file storage (Data Lake)
Apache Spark	PySpark 3.x	Distributed data processing & transformation
Apache Hive	Hive Metastore	SQL query engine & schema registry
MySQL	MySQL 8.0	Relational source database
Apache Sqoop	Sqoop 1.x	JDBC bulk data transfer MySQL → HDFS
Docker	Docker Compose	Container orchestration for all services
Apache Parquet	Columnar format	Storage format for Silver and Gold layers
Python	Python 3.x	PySpark scripts and data quality logic
SQL (HiveQL)	HiveQL	DDL for external tables, Gold layer testing

4. Source Data & MySQL Database Design

4.1 Database Creation (DDL)

The source schema is created in MySQL under the ecommerce database. Each table is designed to reflect the normalised transactional structure of an e-commerce platform. The schema is created using the DDL_ecommerce.sql script.

4.2 Schema Design

Table	Key Columns	Data Types
customers	customer_id (PK)	VARCHAR(50), city, state, zip_code_prefix INT
orders	order_id (PK)	customer_id FK, order_status, 5× DATETIME timestamps
order_items	order_id + item_id	product_id FK, price DECIMAL(10,2), freight_value
reviews	review_id (PK)	review_score INT, comment TEXT, 2× DATETIME
products	product_id (PK)	category VARCHAR, 7× physical dimension INTs

4.3 Data Loading

CSV files are loaded into MySQL using the LOAD DATA INFILE command. Each table is first truncated before loading to ensure idempotency. The data_loaded.sql script manages this process and uses the MySQL secure file path /var/lib/mysql-files/ for file staging.

- Files are first copied into the MySQL Docker container using docker cp
- The ENCLOSED BY '"' option handles quoted CSV fields correctly
- IGNORE 1 ROWS skips the CSV header line
- The reviews table uses the IGNORE keyword to skip duplicate primary keys on load

5. Data Ingestion Layer (Bronze)

The Bronze layer represents raw, unmodified data transferred from MySQL into HDFS. No transformations are applied at this stage — data is preserved exactly as it exists in the source system, including any quality issues such as nulls, duplicates, or incorrect types.

5.1 Transfer Method A — Apache Sqoop

Sqoop provides a production-grade JDBC bridge between MySQL and HDFS. Each table is imported independently with a dedicated mapper. The `sqoop_mysql_cmd_hdfs.sql` file contains all commands.

Table	Target HDFS Path	Notes
customers	/user/root/ecommerce/customers	--m 1 (single mapper)
orders	/user/root/ecommerce/orders	--m 1 (single mapper)
order_items	/user/root/ecommerce/order_items	--split-by order_item_id
reviews	/user/root/ecommerce/reviews	--m 1 (single mapper)
products	/user/root/ecommerce/products	--m 1 (single mapper)

5.2 Transfer Method B — Manual HDFS Put

An alternative manual method bypasses Sqoop by directly placing CSV files into HDFS via the `docker cp` and `hdfs dfs -put` commands. This approach is documented in `move_data_mysql_HDFS_without_sqoop.txt` and is useful when Sqoop is unavailable.

- Docker `cp` copies CSV files from the host machine into the namenode container
- `hdfs dfs -put` transfers files from the namenode filesystem into HDFS
- HDFS directories are pre-created using `hdfs dfs -mkdir -p`

5.3 CSV → Parquet Conversion

After the initial CSV ingestion, the pipeline converts all files to Apache Parquet format for performance and compression benefits. PySpark reads the CSV files and writes them as Parquet using the overwrite mode to ensure repeatability.

Parquet provides columnar storage, predicate pushdown, and significantly smaller file sizes compared to CSV, making it the standard format for all downstream processing layers.

6. Data Cleaning Layer (Silver)

The Silver layer is the data quality engineering stage. PySpark transformations address all identified quality issues while preserving business meaning. The output is clean, typed, and deduplicated Parquet files written to `hdfs://namenode:8020/user/root/silver/`.

6.1 Schema Validation & Type Casting

The pipeline begins with `printSchema()` inspection across all five tables. The reviews table requires explicit type correction — `review_score` is cast from string to integer, and both timestamp fields are parsed using `to_timestamp()`.

6.2 Duplicate Detection & Removal

A systematic deduplication check is performed using `groupBy` on each table's primary key. The `find_duplicates()` utility function standardises this check across all tables.

Table	Dedup Keys	Duplicates Found	Action Taken
customers	customer_id	None	No action required
orders	order_id	None	No action required
order_items	order_id + order_item_id	None	No action required
reviews	review_id	YES	<code>dropDuplicates([review_id])</code>
products	product_id	None	No action required

6.3 Null & NaN Handling

A two-pass null check is performed: first using `isNull()` for all columns, then a combined `isNull() | isnan()` check for numeric columns only. This ensures NaN values in float/double columns are not missed by standard null checks.

Reviews Table — Null Strategy

- Drop rows where `review_id`, `order_id`, or `review_score` is null (primary/foreign key integrity)
- Fill `review_comment_title` nulls with 'no title'
- Fill `review_comment_message` nulls with 'no comment'
- Preserve nulls in `review_creation_date` and `review_answer_timestamp` — these represent real-world event timing gaps

Products Table — Null Strategy

- Fill product_category_name nulls with 'unknown' (meaningful fallback)
- Fill all 7 physical dimension columns (weight, length, height, width, photos, name_length, description_length) with 0

Orders Table — Null Strategy

Null values in orders timestamp columns (e.g. order_delivered_customer_date) are intentionally preserved. These represent orders that are still in-progress, cancelled, or pending — removing them would introduce survivorship bias into delivery time calculations.

6.4 Silver Hive External Tables

All five cleaned tables are registered as Hive external tables in the silver database using the silver_hive.sql script. Each table maps directly to the corresponding HDFS Parquet path, enabling SQL queries without duplicating data.

7. Analytical Layer (Gold)

The Gold layer transforms clean Silver data into a dimensional model optimised for analytical queries and BI reporting. The design follows the Star Schema pattern with one central fact table and four surrounding dimension tables.

7.1 Star Schema Design



7.2 Fact Table — fact_sales

The fact table is built at the grain of one row per order item. It is constructed by joining the orders, order_items, products, and customers Silver tables using left joins anchored on the orders table as the primary business event.

Computed KPI Columns

- delivery_time_days:** `datediff(order_delivered_customer_date, order_purchase_timestamp)` — measures fulfilment speed
- total_price:** `price + freight_value` — true cost to the customer per line item

Column Group	Source Table	Key Columns
Order Context	orders	order_id, order_status, all 5 timestamps
Line Item	order_items	order_item_id, seller_id, price, freight_value
Product Info	products	product_category_name, all physical dimensions
Customer Info	customers	customer_id, city, state, zip_code_prefix
Computed KPIs	Derived	delivery_time_days, total_price

7.3 Dimension Tables

dim_customers

Contains customer profile information: `customer_id`, `customer_unique_id`, `city`, `state`, and `zip code prefix`. The `customer_unique_id` field distinguishes returning customers who may have placed multiple orders under different `customer_ids`.

dim_products

Contains product catalogue attributes used for category analysis and product performance reporting: `product_id`, category name, physical dimensions (weight, length, height, width), and photo count.

dim_reviews

Contains customer satisfaction data: `review_id`, linked `order_id`, `review_score` (1–5), and a derived boolean column `is_positive` (True when `review_score` $\geq$ 4). This column serves as a quick sentiment proxy for BI filtering.

dim_time

Provides a time intelligence layer derived from `order_purchase_timestamp`. Exposes `order_date`, `year`, `month`, and `day` as separate columns to enable time-series analysis, seasonal trends, and month-over-month comparisons in BI tools without requiring date functions in every query.

8. Hive Schema & External Tables

Apache Hive provides the SQL governance layer for both the Silver and Gold zones. All tables are created as EXTERNAL tables — meaning Hive registers the schema metadata in its Metastore but does NOT own or manage the underlying Parquet data files. Deleting a Hive table removes only the metadata, not the data.

8.1 Database Structure

Hive Database	Tables	HDFS Base Path
silver	5 cleaned source tables	hdfs://namenode:8020/user/root/silver/
gold	1 fact + 4 dimension tables	hdfs://namenode:8020/user/root/gold/

8.2 External Table Pattern

Each EXTERNAL TABLE declaration in Hive specifies the column names, data types, storage format (STORED AS PARQUET), and the HDFS LOCATION. This decouples schema governance from physical storage, enabling multiple compute engines (Spark, Hive, Presto) to read the same data simultaneously.

8.3 Validation Queries

After table creation, the gold_test.sql script validates the Gold layer using SELECT COUNT(*), column inspection via DESCRIBE FORMATTED, and row samples via SELECT * LIMIT 10.

9. Data Quality Findings

The Silver layer cleaning process uncovered several data quality issues. The following table summarises all findings and the engineering decisions taken for each.

Table	Issue Type	Severity	Resolution
reviews	Duplicate review_ids	HIGH	dropDuplicates([review_id])
reviews	Wrong data types	HIGH	Cast review_score to INT, parse timestamps
reviews	Null comment fields	MEDIUM	fillna with 'no title' / 'no comment'
reviews	Null critical keys	HIGH	dropna on review_id, order_id, review_score
products	Null category names	MEDIUM	fillna with 'unknown'
products	Null dimension values	LOW	fillna with 0 for all 7 dimension columns
orders	Null timestamps	INTENTIONAL	Preserved — reflect real order states

9.1 Key Engineering Decisions

- Null timestamps in orders are not errors — they represent valid business states (pending, cancelled orders)
- Physical dimension nulls in products are filled with 0 rather than dropped, preventing loss of valid product records
- Duplicate reviews are removed only by primary key — full row inspection confirmed they were true duplicates
- The `get_null_nan_counter()` function was purpose-built to handle type-aware null checking (NaN only applies to float/double columns in Spark)

10. Key Business KPIs Enabled

The Gold dimensional model is designed to support a wide range of business intelligence queries. The following KPIs are directly computable from the Gold layer without any additional transformation.

KPI	Source Tables	Description
Total Revenue by Period	fact_sales + dim_time	SUM(total_price) grouped by year/month
Average Delivery Time	fact_sales	AVG(delivery_time_days) by state/category
Customer Satisfaction Rate	dim_reviews	% of is_positive reviews per order cohort
Revenue by Product Category	fact_sales + dim_products	SUM(price) grouped by product_category_name
Top Cities by Order Volume	fact_sales + dim_customers	COUNT(order_id) grouped by customer_city
Freight Cost Analysis	fact_sales	AVG/SUM freight_value by region/category
Monthly Active Customers	fact_sales + dim_time	COUNT(DISTINCT customer_unique_id) by month
Order Status Distribution	fact_sales	COUNT by order_status for funnel analysis

11. Conclusion & Future Work

11.1 Project Achievements

This project successfully delivers a production-quality, end-to-end data engineering pipeline covering the complete lifecycle from raw relational data to analytics-ready dimensional tables. All pipeline stages — ingestion, cleaning, transformation, and schema governance — are implemented with industry-standard tools and documented with reproducible scripts.

- Implemented a full Medallion Architecture on Hadoop infrastructure inside Docker
- Developed reusable PySpark utility functions for schema inspection, duplicate detection, and null analysis
- Applied type-aware data quality checks distinguishing NULL vs NaN for numeric columns
- Built a 5-table star schema with computed KPI columns ready for BI tool connection
- Documented two alternative data transfer methods (Sqoop vs manual HDFS put) with full commands
- Created all Hive external tables with correct STORED AS PARQUET and LOCATION declarations

11.2 Technical Skills Demonstrated

Domain	Skills Applied
Data Engineering	Pipeline design, ETL/ELT, Medallion Architecture
Distributed Computing	Apache Spark, HDFS, YARN cluster management
Data Quality	Deduplication, null strategies, type validation
Data Modelling	Star schema, fact/dimension design, grain selection
SQL & HiveQL	DDL, external tables, schema governance
DevOps	Docker, containerisation, service orchestration

11.3 Recommended Future Enhancements

- Orchestration: Integrate Apache Airflow to schedule and monitor pipeline DAGs automatically
- Streaming: Add Apache Kafka + Spark Structured Streaming for real-time order ingestion
- BI Integration: Connect Power BI or Apache Superset directly to the Gold Hive tables

- Data Catalogue: Implement Apache Atlas for metadata management and data lineage tracking
- ML Layer: Build a predictive delivery delay model using the `delivery_time_days` feature in `fact_sales`
- Incremental Loading: Replace full overwrite mode with CDC (Change Data Capture) using Delta Lake

Information Technology Institute (ITI) — 9-Month Professional Diploma

Big Data Track | Graduation Project | April 2026